

Digital Image Analysis Tutorial 4

```
1  """AI531C T04 studio – Noise Fingerprint Lab (Tue 25 Aug; COMPACT 25-min studio –
2  Assessment Test I occupies the first half of this slot per the lesson plan).
3  INPUT: camera; three corrupted versions generated with a FIXED seed.
4  OPERATION: for each mystery image: flat-patch histogram -> identify the noise ->
5  | estimate its parameter -> check against truth.
6  PARAMETERS: seed 11; Gaussian sigma 18; salt&pepper 4%; Poisson scale 12.
7  EXPECTED OUTPUT: printed estimates (asserted near truth) + expected outputs/T04_panel.png.
8  INTERPRETATION: the flat patch is the stethoscope – diagnosis before treatment (L11 next week).
9  """
10 import numpy as np, cv2, os, matplotlib
11 matplotlib.use("Agg"); import matplotlib.pyplot as plt
12 from skimage import data
13 HERE = os.path.dirname(os.path.abspath(__file__))
14 os.makedirs(f"{HERE}/expected_outputs", exist_ok=True); os.makedirs(f"{HERE}/inputs", exist_ok=True)
15 rng = np.random.default_rng(11)
16 img = data.camera().astype(np.float64)
17 flat = (slice(20, 60), slice(150, 350)) # sky strip: flat-ish
18
19 A = np.clip(img + rng.normal(0, 18, img.shape), 0, 255) # gaussian
20 B = img.copy(); m = rng.random(img.shape); B[m < 0.02] = 0; B[m > 0.98] = 255 # s&p 4%
21 C = rng.poisson(img / 12.0) * 12.0 # poisson
22 for name, x in zip("ABC", (A, B, C)): cv2.imwrite(f"{HERE}/inputs/mystery_{name}.png", np.clip(x,0,255).astype(np.uint8))
23
24 sigA = A[flat].std(); fracB = ((B[flat] == 0) | (B[flat] == 255)).mean()
25 darkC = C[img < 60].std(); brightC = C[img > 180].std()
26 print(f"[1] mystery A flat-patch std = {sigA:.1f} (truth: Gaussian sigma 18)")
27 print(f"[2] mystery B extreme-pixel fraction = {fracB:.3f} (truth: s&p 4% total)")
28 print(f"[3] mystery C: std in DARK regions {darkC:.1f} vs BRIGHT {brightC:.1f} – signal-dependent!")
29 assert abs(sigA - 18) < 3 and 0.02 < fracB < 0.06 and brightC > darkC
30
31 fig, ax = plt.subplots(2, 3, figsize=(12.5, 6))
32 for j, (x, t) in enumerate(zip((A, B, C), ("mystery A", "mystery B", "mystery C"))):
33     ax[0, j].imshow(x, cmap="gray", vmin=0, vmax=255); ax[0, j].set_title(t, fontsize=13); ax[0, j].axis("off")
34     ax[1, j].hist(x[flat].ravel(), bins=60, color="#17365D"); ax[1, j].set_title("flat-patch histogram", fontsize=12)
35 plt.tight_layout(); plt.savefig(f"{HERE}/expected_outputs/T04_panel.png", dpi=120); plt.close()
36 print("saved expected_outputs/T04_panel.png")
37
38 def support():
39     print("[SUPPORT] identify only mystery B: print the fraction of pure-black and pure-white")
40     print(" pixels in the flat strip. Which classical noise makes pixels JUMP to the extremes?")
41
42 def extension():
43     med = cv2.medianBlur(np.clip(B,0,255).astype(np.uint8), 3).astype(np.float64)
44     print(f"[EXTENSION] 3x3 median on mystery B: flat-strip extreme fraction {(med[flat]==0)|(med[flat]==255).mean():.4f}")
45     print(" One week EARLY: why did the median annihilate this noise? (L11 explains.)")
46
47 if __name__ == "__main__":
48     import sys
49     if len(sys.argv) > 1 and sys.argv[1] == "support": support()
50     elif len(sys.argv) > 1 and sys.argv[1] == "extension": extension()
```

Output Images:

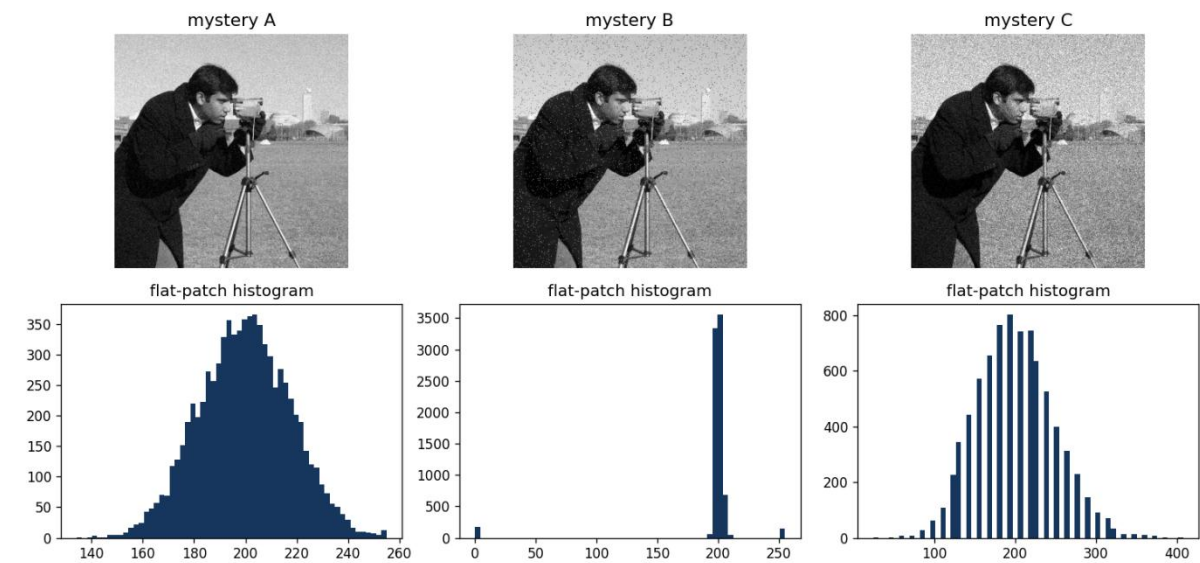


Image A

Image A represents the Gaussian Noise

Image B

Image B represents the Salt and Pepper Noise